

Python Basics

Complete Study Notes

Source: 1_basic_of_python.pdf (50 slides)

The Easylearn Academy

Covers: Introduction → Variables → Data Types → Operators

26 Chapters + Final Exam Revision

Every chapter ends with a Quick Revision box.

Boxes marked ⚠ flag errors or outdated content found in the original PDF.

Python Basics — Complete Study Notes

Source: [1_basic_of_python.pdf](#) (50 slides) — *The Easylearn Academy* **Covers:** Introduction → Comments → Variables → Keywords → Input → Data Types (Number, String, List, Tuple, Dictionary, Set) → Multi-line statements → Operators → Operator Precedence

How to use these notes: Read top to bottom the first time. Before an exam, read only the **Quick Revision** boxes at the end of each chapter, then the ★ **FINAL EXAM REVISION** section at the very end.

Wherever the PDF contains a mistake, an outdated rule, or a confusing sentence, you will see a ⚠ **PDF Check** box. Do not skip these — examiners often ask exactly these points.

Table of Contents

#	Chapter
1	Introduction to Python
2	History of Python
3	Advantages of Python
4	Features of Python
5	Applications of Python
6	Getting Python (Installation & Documentation)
7	First Python Program
8	Comments
9	Quotations (String Literals)
10	Multiple Statements on a Single Line
11	Variables
12	Printing Messages with Variables
13	More About Variables
14	Python Keywords
15	Variable Naming Rules
16	Taking Input from the User
17	Data Types (Mutable vs Immutable)
18	Numeric Data Types
19	Strings
20	Lists
21	Tuples
22	Dictionaries
23	Sets
24	Multi-Line Statements
25	Operators
26	Operator Precedence
★	Final Exam Revision

CHAPTER 1: Introduction to Python

1.1 What is Python?

Definition: Python is a **general-purpose, interpreted, interactive, object-oriented and high-level** programming language.

Simple Meaning: Python is a language used to give instructions to a computer. It can be used to build almost any kind of software, and it is written in a style that looks close to normal English.

Breaking down the definition (very important for 3-5 mark answers)

Term	Meaning in simple words
General-purpose	Not made for one job only. You can build websites, games, AI, desktop apps, anything.
Interpreted	The code is run line by line by a program called the <i>interpreter</i> . You do not have to compile the whole program first.
Interactive	You can type one line in the Python shell and immediately see its result. Good for testing small things.
Object-oriented	It supports classes and objects — a way of grouping data and functions together.
High-level	It is close to human language and far from machine language, so you don't manage memory or hardware yourself.

Other introduction points from the PDF

- Python is **easy to learn and easy to use**.
- Python source code is available under the **GNU General Public License (GPL)**.
- The **latest version mentioned in the PDF is Python 3.14** (at the time the presentation was last updated).

⚠ **PDF Check — Licence** The PDF says Python's source code is available under the **GNU GPL**. In reality, Python is released under the **Python Software Foundation License (PSF License)**, which is a permissive open-source licence. The GPL claim is a very common textbook error. It is safest to write: "*Python is free and open source software.*" If your syllabus/paper strictly follows this PDF, write GPL — but know the correct fact.

1.2 Overview — Why Python Matters

- Python can be used to develop **large-sized applications** such as **banking software** and **ERP applications**. (*ERP = Enterprise Resource Planning — big business software that handles accounts, stock, HR, etc.*)
- It **uses English keywords frequently**, so the language is easy to understand.
- Python is a **great language for beginner-level programmers**.

In Simple Words: Python is easy enough for a first-time learner, but powerful enough for banks and big companies to run real business software on it.

Quick Revision — Chapter 1

- Python = general-purpose + interpreted + interactive + object-oriented + high-level.
- Used for big applications like banking software and ERP.
- Uses English-like keywords → easy to read.
- Best for beginners.
- PDF says GPL licence (real answer: PSF licence).
- Latest version in PDF: **Python 3.14**.

CHAPTER 2: History of Python

Definition: Python was developed by **Guido van Rossum** during **1980-1990** at the **National Research Institute for Mathematics and Computer Science (CWI)** in the **Netherlands**.

Key historical facts

Point	Detail
Creator	Guido van Rossum
Period	1980 - 1990
Place	National Research Institute for Mathematics and Computer Science, Netherlands
Name origin	Named after the comedy TV show Monty Python's Flying Circus — NOT after the Python snake
Inspired by	ABC, Modula-3, C, C++, Algol-68, SmallTalk, Unix shell and other scripting languages

Remember: The single most-asked history question is "*Python is named after which show?*" → **Monty Python's Flying Circus**, not the snake.

Note on "inspired": The PDF explains inspiration as "*learned from mistake*" — meaning Guido looked at the good and bad parts of older languages and kept only the good ideas in Python.

Quick Revision — Chapter 2

- Guido van Rossum → 1980-1990 → Netherlands (CWI).
- Named after Monty Python's Flying Circus.
- Inspired by ABC, Modula-3, C, C++, Algol-68, SmallTalk, Unix shell.

CHAPTER 3: Advantages of Python

These 8 points are a classic **5-mark question**. Learn the keyword of each point.

#	Advantage	Simple explanation
1	Readable	Python code looks clean and is easy to read, even for someone new.
2	Easy to Learn	It is an expressive, high-level language, so less code does more work.
3	Cross platform	The same Python program runs on Mac, Windows, Linux, Unix, etc.
4	Open Source	Its source code is openly available; anyone can view and improve it.
5	Large standard library	Python already includes many ready-to-use functions, so you don't write everything yourself.
6	Free	Free to download and free to use.
7	Supports exception handling	Python can catch run-time errors, so the program does not stop suddenly.
8	Automatic memory management	Memory is cleaned and freed automatically. You never have to free memory by hand (unlike C/C++).

In Simple Words: Python is free, works everywhere, comes with a lot of ready-made tools, cleans up its own memory, and doesn't crash the moment something goes wrong.

Remember: *Open Source* means the code is public. *Free* means it costs nothing. In exams, write both separately — they are two different points.

Quick Revision — Chapter 3

Memory trick — **R E C O L F E A**: **R**eadable, **E**asy to learn, **C**ross-platform, **O**pen source, **L**arge library, **F**ree, **E**xception handling, **A**utomatic memory management.

CHAPTER 4: Features of Python

From the PDF's "Python Features..." slide:

1. **Supports POP as well as OOP** techniques. - **POP** = Procedure Oriented Programming — the program is written as a set of functions/procedures (like in C). - **OOP** = Object Oriented Programming — the program is written using classes and objects (like in C++/Java). - Python allows **both** styles.
2. It can be used as a **scripting language**, or **compiled to byte-code** for building large applications. - *Byte-code* = a middle-level code (`.pyc` files) that Python creates internally so the program starts faster next time.
3. In Python we can **store any type of value in any variable**. - You don't declare the type. The same variable can hold a number now and a string later.
4. It can be **easily integrated with other programming languages** (like C, C++, Java).

Example of point 3:

```
x = 10          # x holds an integer
x = "hello"     # the SAME variable now holds a string – allowed in Python
```

Remember: This "any value in any variable" behaviour is called **dynamic typing**. In C/C++/Java it is not allowed; in Python it is.

Quick Revision — Chapter 4

- POP + OOP both supported.
 - Scripting language + can be compiled to byte-code.
 - Any type of value in any variable (dynamic typing).
 - Easy integration with other languages.
-

CHAPTER 5: Applications of Python

5.1 Where Python Can Be Used (7 areas from the PDF)

#	Area	What it means
1	Web development	Frameworks like Django and Flask are built on Python. They help you write server-side code: managing the database, backend logic, URL mapping, etc.
2	Machine learning	Writing logic so that a machine can learn and solve a problem on its own. Example: product recommendations on Amazon, Flipkart, eBay, which learn a user's interest.
3	Data Analysis	Analysing data and showing it visually as charts.
4	Scripting	Writing small programs to automate simple tasks, e.g. sending automated reply emails.
5	Game development	Games can be built in Python.
6	Embedded applications	Programs that run inside small devices/hardware.
7	Desktop applications	Built using libraries like TKinter or QT .

5.2 What Can Python Do? (from the PDF)

- Python can be used **on a server to create web applications**.
- Python can be used **together with software to create workflows**.
- Python can **connect to database systems**. It can also **read and modify files**.
- Python can **handle big data** and perform **complex mathematics (data mining)**.
- Python can be used for **rapid prototyping** or for **production-ready software development**.

In Simple Words: *Rapid prototyping* = quickly building a rough working version to test an idea. *Production-ready* = the final, polished software that real users will use. Python is good at both.

Quick Revision — Chapter 5

- Web (Django, Flask) • Machine Learning • Data Analysis • Scripting • Games • Embedded • Desktop (TKinter, QT).
- Server-side web apps, workflows, database connection, file read/modify, big data, data mining, prototyping and production software.

CHAPTER 6: Getting Python — Setup and Documentation

- Python is available on a **wide variety of platforms**, including **Linux and Mac OS X**.
- To check if Python is already installed: **open a terminal window and type `python`**. It will show whether it is installed and which version.
- The most up-to-date source code, binaries, documentation and news are available on the **official website**:
<https://www.python.org/>
- Documentation can be downloaded from: **<https://www.python.org/doc/>**
- Python's documentation is available in **HTML, PDF and PostScript** formats.

Remember: Official website → [python.org](https://www.python.org/). Documentation formats → **HTML, PDF, PostScript** (this exact 3-item list is a favourite 2-mark question).

CHAPTER 7: Your First Python Program

The PDF's first program:

```
'''
Hello world program @ the easylearn academy
Author : Ankit Patel
Date : today
'''
print("Hello Student")
# Second print statement
print("We are going to learn python @ the easylearn academy")
```

Expected Output:

```
Hello Student
We are going to learn python @ the easylearn academy
```

Line-by-line explanation

Line	What it does
''' ... '''	A multi-line comment giving program name, author and date. Python ignores it completely.
print("Hello Student")	<code>print()</code> displays whatever is inside the brackets on the screen.
# Second print statement	A single-line comment . Ignored by Python.
print("We are going...")	Displays the second message on the next line (each <code>print()</code> moves to a new line by default).

Remember: `print()` automatically moves to a new line after printing. To stay on the same line you must use `end=''` (explained in Chapter 12).

CHAPTER 8: Comments

Definition: A comment is text inside a program that **does not affect the outcome of the code**. It is written only to tell a reader what has been done in the program or in a block of code.

Simple Meaning: Comments are notes for humans. Python skips them completely while running.

8.1 Why comments are useful

- When **someone else has written the code** and you are reading it for **bug fixing** or to **change the logic**, comments tell you what each part does.
- They remind *you* what you did when you return to your own code months later.

8.2 Types of Comments in Python

1. Single-line comment —

```
# This is just a comment. Anything written here is ignored by Python
print("Hi") # a comment can also sit after code on the same line
```

Everything after the **#** **on that line only** is ignored.

2. Multi-line comment — ''' or """

```
'''
This
is a
multi-line
comment
'''
```

Everything between the two sets of triple quotes is ignored.

Comparison Table

Point	Single-line comment	Multi-line comment
Symbol	#	''' ''' or """ """
Covers	Only the rest of that one line	Many lines at once
Use	Short note about one statement	Program header, author, date, long explanation
Example	# add two numbers	''' Author: Ankit '''

Remember: Two types of comments in Python: **single-line (#)** and **multi-line (''')**. This is a very common 2-mark question.

Small technical clarification (useful, not contradicting the PDF): Technically, `'''...'''` is a **string** that Python creates and then throws away because it isn't stored in any variable. That is why it behaves like a comment. When it is placed as the first line inside a function or class, it is called a **docstring** (documentation string).

CHAPTER 9: Quotation in Python

Python accepts **three kinds of quotes** to write string literals:

Quote type	Symbol	Used for
Single	' '	Normal string
Double	" "	Normal string
Triple	' ' ' ' ' ' or " " " " " "	Multi-line string (paragraph)

The one golden rule: the **same type of quote must start and end** the string.

Example from the PDF

```
print('Python')
print("Python is general purpose script")
print("""This is a paragraph.
It is made up of multiple lines and sentences. """)
```

Expected Output:

```
Python
Python is general purpose script
This is a paragraph.
It is made up of multiple lines and sentences.
```

In Simple Words: Single and double quotes are the same for Python. Triple quotes are special because your text can continue over several lines.

⚠ **PDF Check — Curly quotes** Several slides in the PDF show **curly/smart quotes** like `'Python'` and `"Python"` (because the slides were typed in PowerPoint). **These will NOT work in Python.** You must always type **straight quotes**: `'` and `"`. Writing `print('Python')` gives a `SyntaxError`. Keep this in mind while copying any example from the PDF.

Remember: Triple quotes are used for a **multi-line string**. Same quote must open and close a string.

CHAPTER 10: Multiple Statements on a Single Line

- The **semicolon** `;` allows you to write **multiple statements on a single line**.

Example:

```
print("Hi"); print("I am learning Python")
```

Expected Output:

```
Hi  
I am learning Python
```

What CANNOT be written this way

The PDF lists four things that cannot be combined on a single line using semicolons:

1. **Decision making statements** (`if`, `elif`, `else`)
2. **Loops** (`for`, `while`)
3. **Functions** (`def`)
4. **Classes** (`class`)

In Simple Words: A semicolon can only join **simple, one-line statements**. Anything that opens a block with a colon `:` and needs indented lines under it (if, loops, functions, classes) cannot be squeezed onto one line with semicolons.

⚠ **PDF Check — Confusing wording** The slide says: "*The semicolon (;) allows multiple statements on the single line but it is not possible.*" That sentence reads like a contradiction. The intended meaning is: "*The semicolon allows multiple statements on a single line, but this is not possible for decision-making statements, loops, functions and classes.*" Write it that way in your exam.

Remember: Semicolon works only for **simple statements**. Blocks (`if`, `for`, `while`, `def`, `class`) need proper indentation on separate lines.

CHAPTER 11: Creating Variables

Definition: A variable is a **named location in main memory** where we can store data temporarily.

Simple Meaning: A variable is like a labelled box in the computer's memory. You put a value inside and use the label whenever you need that value.

11.1 Every variable has three things

Part	Meaning
Name	The label you use to refer to it (e.g. <code>age</code>)
Value	The data stored inside it (e.g. <code>21</code>)
Location	The unique memory address where it is stored

Each variable takes some **memory space (in bytes)** depending on the **type of value** assigned to it.

11.2 How to create a variable

Syntax:

```
variable_name = value
```

- Variable **name on the left** of `=`
- **Value on the right** of `=`
- `=` here means **"store this value into this variable"** (it is NOT the maths "equal to").

Examples from the PDF

```
Name = "The easylearn academy" # string
Year = 2021                    # integer
Weight = 80.25                 # float
Gender = True                  # boolean
```

11.3 No type declaration needed

- You **do not have to mention the type** of the variable.
- **Python guesses the type automatically** based on the value stored in it.

Example:

```
a = 5          # Python decides: int
b = 5.0        # Python decides: float
c = "5"        # Python decides: str
d = False      # Python decides: bool
```

Remember: In C you write `int a = 5;`. In Python you just write `a = 5`. Python figures out the type itself — this is called **dynamic typing**.

CHAPTER 12: How to Print a Message Using a Variable

Printing only the variable

```
name = "the easylearn academy"
print(name)          # display only variable
```

Output:

```
the easylearn academy
```

Printing a message and a variable together

```
print("you are learning python at ", name)
```

Output:

```
you are learning python at  the easylearn academy
```

Here the **comma** , separates the two items. Python automatically puts a **space** where the comma is.

Printing the next message on the SAME line — `end=''`

The PDF's point: "To print the next message on the same line, use the second argument in the print method. It should be `end=''`."

Syntax:

```
print("text", end='')
```

Example:

```
print("Hello ", end='')
print("Students")
```

Output:

```
Hello Students
```

In Simple Words: By default `print()` jumps to a new line after printing. `end=''` tells Python: "don't jump to a new line, end this print with nothing." You can also use `end=' '` (a space) or `end='- '` etc.

⚠ **PDF Check** The slide mentions `end=''` but the example given below it does not actually use it. Also, `end` is a **keyword argument**, not simply "the second argument" — you must write the word `end=` explicitly. The example above shows the correct usage.

Remember: `print()` → new line by default. `print(..., end='')` → stay on the same line.

CHAPTER 13: More About Variables

Six important rules/facts from the PDF:

1. You can **change a variable's value anywhere** in your program.
2. **Variable names are case sensitive** — `age`, `Age` and `AGE` are three different variables.
3. **All variables are deleted** when the program finishes, or stops in between, or if the computer restarts while your program is running.
4. Variables are used to store **input, intermediate results and final results**.
5. Variables are also used in **expressions** (e.g. `total = price * qty`).
6. A variable name **can be anything, but it cannot be a keyword**.

Example of point 1 and 5:

```
price = 100      # initial value
price = 150      # value changed later – allowed
qty = 2
total = price * qty  # variable used in an expression
print(total)
```

Output:

```
300
```

In Simple Words (point 3): Variables live in **RAM (main memory)**, which is temporary. When the program ends, the memory is released and everything stored in it is gone. To keep data permanently you must save it in a **file or database**.

Remember: Variables are **temporary storage**. Keywords **cannot** be used as variable names.

CHAPTER 14: Python Keywords

Definition: A Python keyword is a **reserved word** that you **cannot use as the name** of your variable, class, function, etc. These words have a **special meaning** and are used for special purposes in Python.

Simple Meaning: Keywords are words that Python has already booked for itself. You are not allowed to use them as your own names.

14.1 Important points

- Keywords are used to **define the syntax and structure** of the Python language.
- In Python, **all keywords are case sensitive** — so be careful while using them.
- Example:** `class` is used to create a class, so you cannot name a variable `class` — it will cause an error.
- There are a total of **35 keywords** in Python as of now.

Example of the error:

```
class = 5      # ✗ SyntaxError – 'class' is a keyword
Class = 5     # ✓ allowed, because keywords are case sensitive ('Class' ≠ 'class')
```

14.2 How to see the keyword list on your own computer

The PDF's steps:

- Open **Command Prompt** (Terminal on Mac OS).
- Type `python` and press Enter.
- Type `keywords` and press Enter to get the keyword list for the Python version installed on your system.

⚠ **PDF Check — This will not work as written** Typing plain `keywords` inside the Python shell gives a `NameError`. The correct ways are:

```
>>> help("keywords")
```

or

```
>>> import keyword
>>> print(keyword.kwlist)
>>> print(len(keyword.kwlist))    # 35
```

(The PDF's step probably meant `help("keywords")`.)

14.3 List of 35 Keywords

False	class	from	or
None	continue	global	pass
True	def	if	raise
and	del	import	return
as	elif	in	try
assert	else	is	while
async	except	lambda	with
await	finally	nonlocal	yield
break	for	not	

Total = 35

Small memory aid — group them by job

Group	Keywords
Values	<code>True</code> , <code>False</code> , <code>None</code>
Conditions	<code>if</code> , <code>elif</code> , <code>else</code>
Loops	<code>for</code> , <code>while</code> , <code>break</code> , <code>continue</code> , <code>pass</code>
Functions	<code>def</code> , <code>return</code> , <code>lambda</code> , <code>yield</code>
Classes / objects	<code>class</code>
Error handling	<code>try</code> , <code>except</code> , <code>finally</code> , <code>raise</code> , <code>assert</code>
Modules	<code>import</code> , <code>from</code> , <code>as</code>
Logic	<code>and</code> , <code>or</code> , <code>not</code> , <code>in</code> , <code>is</code>
Scope	<code>global</code> , <code>nonlocal</code> , <code>del</code>
Others	<code>with</code> , <code>async</code> , <code>await</code>

Remember: 35 keywords, all **case sensitive**, and `True`, `False`, `None` **start with a capital letter** — this catches many students out.

CHAPTER 15: Naming Rules for Variables

15.1 Rules (compulsory — breaking them causes an error)

1. A variable name can contain only **alphabets, numbers and underscores** (A-Z, a-z, 0-9 and `_`).
2. A variable name **must start with an alphabet or an underscore** `_`.
3. A variable name **must not start with a number**.
4. Variable names are **case-sensitive** — `age`, `Age`, `AGE` are three different variables.
5. A variable name **cannot be a keyword** (from Chapter 14).

15.2 Conventions (good habits — not errors, but expected)

- Give variable names in **lowercase**.
- If the name has **multiple words, separate them with an underscore** `_` — `car_name` is easier to read than `carname`.
- Give **meaningful names** like `age`, `car_name`. **Avoid** meaningless names like `x` and `y`.

15.3 Valid vs Invalid — quick table

Name	Valid?	Reason
<code>age</code>	✓	Starts with a letter
<code>_total</code>	✓	Starting underscore is allowed
<code>car_name</code>	✓	Letters + underscore
<code>student1</code>	✓	Number allowed, just not at the start
<code>1student</code>	✗	Starts with a number
<code>car name</code>	✗	Space not allowed
<code>car-name</code>	✗	Hyphen <code>-</code> not allowed
<code>class</code>	✗	It is a keyword
<code>total\$</code>	✗	Special symbol not allowed

Tiny clarification: The PDF writes the allowed range as "**A-z**". Read this as **A-Z and a-z** (uppercase and lowercase letters). Symbols like `[`, `\`, `]`, `^`, ``` that fall between `z` and `a` in the character table are **not** allowed.

Remember: Start with a **letter or underscore**, never a number, never a keyword, no spaces or special symbols.

CHAPTER 16: Taking Input from the User

Definition: To take input from the user through the keyboard, the `input()` function is used in Python 3.

Syntax:

```
variable = input("input message")
```

- The text inside `input(...)` is the **message (prompt)** shown to the user.
- Whatever the user types is **stored in the variable**.

Example:

```
name = input("Enter your name: ")
print("Hello ", name)
```

Sample run:

```
Enter your name: Pj
Hello Pj
```

⚠ The most important point about `input()`

The PDF says: "Input function can accept any type of value from user."

This needs a clarification, and it is very commonly asked in exams:

- The user can **type** anything (numbers, letters, symbols) — that part is true.
- **BUT `input()` always returns the value as a STRING (`str`).**
- If you need a number, you must **convert** it using `int()` or `float()`.

Example showing the problem:

```
a = input("Enter a number: ") # user types 5
b = input("Enter a number: ") # user types 3
print(a + b)
```

Output:

```
53
```

Because `a` and `b` are the **strings** `"5"` and `"3"`, `+` joins them instead of adding.

Correct version:

```
a = int(input("Enter a number: "))
b = int(input("Enter a number: "))
print(a + b)
```

Output:

```
8
```

Remember: `input()` → always gives a **string**. Use `int(input(...))` for whole numbers and `float(input(...))` for decimal numbers.

Quick Revision — Chapters 11-16

- Variable = named memory location. Has **name, value, location**.
- No type declaration — Python decides the type.
- 35 keywords, case sensitive, cannot be used as names.
- Names: letters/digits/`_`, must not start with a digit, case sensitive, meaningful, lowercase with `_`.
- `variable = input("message")` — always returns a **string**; convert with `int()` / `float()`.

CHAPTER 17: Data Types in Python

Definition: A data type defines the **type of data**. For example, `123` is integer data, while `"hello"` is String data.

17.1 The two categories

Category	Meaning	Types included
Immutable	Part of the value cannot be changed, but the whole value can be replaced.	1. Numbers 2. String 3. Tuple
Mutable	Part of the value can be changed.	1. List 2. Dictionaries 3. Sets

In Simple Words: - **Mutable** = editable. You can change one item inside it without creating a new one. - **Immutable** = fixed. You cannot edit one part; you can only throw away the whole thing and assign a new value.

Example — Immutable (String):

```
s = "hello"
s[0] = "H"      # x ERROR – cannot change part of a string
s = "Hello"     # ✓ allowed – the WHOLE string is replaced
```

Example — Mutable (List):

```
lst = [1, 2, 3]
lst[0] = 100    # ✓ allowed – part of a list can be changed
print(lst)
```

Output:

```
[100, 2, 3]
```

Comparison Table — Mutable vs Immutable

Point	Immutable	Mutable
Meaning	Part of the value cannot be changed	Part of the value can be changed
Whole replacement	Allowed	Allowed
Item-wise change	Not allowed	Allowed
Types	Numbers, String, Tuple	List, Dictionary, Set
Example	<code>s[0] = 'H' → error</code>	<code>lst[0] = 100 → works</code>

Remember (exam gold): Immutable → Number, String, Tuple. Mutable → List, Dictionary, Set. Memorise this as "*NST is fixed, LDS can change.*"

CHAPTER 18: Numeric Data Types

- ```
a = 10
b = 99999999999999999999999999999999 # perfectly fine in Python
```

- ```
weight = 80.25
pi = 3.14
```

```
a = 10      # int
b = 10.5    # float
print(type(a))
print(type(b))
```

```
<class 'int'>
<class 'float'>
```

CHAPTER 19: Python Strings

Definition: A **String** is a **sequence of characters** in Python. The data type of a String in Python is called **str**.

Simple Meaning: A string is text — letters, digits and symbols kept inside quotes.

19.1 Key properties from the PDF

1. Python allows **single quotes**, **double quotes** or **triple double quotes** for strings.
2. **Parts of a string can be accessed using the slice operator** `[ ]` and `[ : ]`.
3. Indexes **start at 0** at the beginning of the string, and the last index is **size – 1** (where *size* is the length of the string).
4. The **plus + sign is the concatenation operator** (joins strings).
5. The **asterisk * is the repetition operator** (repeats the string).
6. Python strings are **immutable** — part of the string **cannot** be replaced, but the **whole** string can be replaced.

19.2 Understanding indexing

For `word = 'Hello World!'`:

Character	H	e	l	l	o	(space)	W	o	r	l	d	!
Index	0	1	2	3	4	5	6	7	8	9	10	11

Size = 12 → last index = 12 – 1 = **11**.

Slicing syntax:

```
string[start : end]
```

- **start** is **included**
- **end** is **NOT included** (it stops just before **end**)
- Missing **start** means "from the beginning"; missing **end** means "till the end"

19.3 The PDF's String Program

```
word = 'Hello World!'

print(word)           # Prints complete string
print(word[0])        # Prints first character of the string
print(word[0:5])      # Prints characters from index 0 up to (not including) 5
print(word[2:])       # Prints string starting from the 3rd character
print(word * 2)       # Prints string two times
print(word + " TEST") # Prints concatenated string
```

Expected Output:

```
Hello World!
H
Hello
llo World!
Hello World!Hello World!
Hello World! TEST
```

The immutability example

```
Word = "the easylearn academy"
Word[0] = 'T'    # x ERROR – will not work (strings are immutable)
```

What happens: `TypeError: 'str' object does not support item assignment`

The correct way (replace the whole string):

```
Word = "The easylearn academy"    # ✓ replacing the entire string is allowed
```

⚠ **PDF Check — Three issues on this slide** 1. The comment `# Prints characters starting from 0th to 2nd position for word[0:5]` is **wrong**. `word[0:5]` prints index **0 to 4** (five characters: **Hello**). The end index is always excluded. 2. The output column shows **The easylearn academy** next to the `Word = "the easylearn academy"` line. This is misleading — that line only **stores** the string, it does not print anything. Also **easylearn** is a typo for **easylearn**. 3. The line ends with a semicolon (;) — legal in Python but unnecessary; don't copy that style.

Remember: - `+` joins strings, `*` repeats them. - Index starts at **0**, last index is **size – 1**. - In `[start:end]`, **end is excluded**. - Strings are **immutable** — you cannot change one character.

Quick Revision — Strings

Operation	Code	Result for <code>word = 'Hello World!'</code>
Full string	<code>word</code>	<code>Hello World!</code>
First character	<code>word[0]</code>	<code>H</code>
Slice	<code>word[0:5]</code>	<code>Hello</code>
From index 2 to end	<code>word[2:]</code>	<code>llo World!</code>
Repeat	<code>word * 2</code>	<code>Hello World!Hello World!</code>
Concatenate	<code>word + " TEST"</code>	<code>Hello World! TEST</code>
Change one character	<code>word[0] = 'h'</code>	Error (immutable)

CHAPTER 20: Python Lists

Definition: A **List** is a special type of variable in which we can **store multiple values**, written inside **square brackets** `[]` and separated by **commas**.

Simple Meaning: A list is one variable that holds many values, like a row of numbered boxes.

20.1 Key properties from the PDF

1. Each value in a list has a **unique positive integer position (index) that starts with 0**.
2. We can **access, modify and delete** any value using the **list name + the value's position**.
3. A list contains items **separated by commas**, created using **brackets** `[]`.
4. Lists are **similar to arrays in C**.
5. A list is a **mutable** data type — we can change or remove any value at any time.
6. Items in a list can be of **different data types**.
7. Values can be accessed using the **slice operator** (`[]` and `[:]`).
8. First index is **0**, last index is **size – 1**.
9. `+` is the **list concatenation operator**; `*` is the **repetition operator**.

20.2 Difference from a C array (important)

Point	Array in C	List in Python
Data types inside	All items must be the same type	Items can be of different types
Size	Fixed at declaration	Can grow or shrink anytime
Declaration	Type must be declared	No type declaration

20.3 The PDF's List Program

```
list = ['Ankit', 123, 2.23, 'Patel', True]
tinylist = [123, "The EasyLearn Academy"]

print(list)           # Prints complete list
print(list[0])        # Prints first element of the list
print(list[1:3])      # Prints elements at index 1 and 2
print(list[2:])       # Prints elements from index 2 to the end
print(tinylist * 2)   # Prints list two times
print(list + tinylist) # Prints concatenated lists
```

Expected Output:

```
['Ankit', 123, 2.23, 'Patel', True]
Ankit
[123, 2.23]
[2.23, 'Patel', True]
[123, 'The EasyLearn Academy', 123, 'The EasyLearn Academy']
['Ankit', 123, 2.23, 'Patel', True, 123, 'The EasyLearn Academy']
```

Notice how the list mixes a **string, an integer, a float and a boolean** — this proves point 6 above.

⚠ **PDF Check — Two points** 1. The comment `print(list[1:3]) # Prints elements starting from 1st till 3rd` is misleading. It prints **index 1 and index 2 only** — the end index 3 is **excluded**. 2. Naming a variable `list` **hides Python's built-in `list()` function**. It works, but it is bad practice. Use `my_list` or `students` instead. (The same applies to `tuple`, `str`, `dict`, `set` used as variable names in later slides.)

20.4 List Methods / Functions

Method	What it does
<code>append(item)</code>	Adds one element to the end of the list
<code>extend(list)</code>	Adds a set of values (another list) at the end of the list
<code>insert(position, item)</code>	Inserts an item at the defined position
<code>remove(item)</code>	Removes the given item from the list
<code>pop(position)</code>	Removes and returns the element at the given position
<code>clear()</code>	Removes all items from the list
<code>index(item)</code>	Returns the index of the first matched item
<code>count(item)</code>	Returns how many times the given item appears
<code>sort()</code>	Sorts items in ascending order (only if all items are of the same type)
<code>reverse()</code>	Reverses the order of items in the list
<code>copy()</code>	Returns a shallow copy of the list

Simple Meaning of "shallow copy": a new list is created, but the items inside are still shared with the original list rather than being duplicated deeply.

Example using the methods

```
marks = [50, 70, 60]

marks.append(90)      # [50, 70, 60, 90]
marks.insert(1, 55)   # [50, 55, 70, 60, 90]
marks.remove(60)      # [50, 55, 70, 90]
print(marks.pop(0))   # prints 50, list becomes [55, 70, 90]
marks.sort()          # [55, 70, 90]
marks.reverse()        # [90, 70, 55]
print(marks)
print(marks.count(70)) # 1
print(marks.index(70)) # 1
```

Expected Output:

```
50
[90, 70, 55]
1
1
```

Remember: - `append()` adds **one** item; `extend()` adds **many** items. - `remove()` needs the **value**; `pop()` needs the **position**. - `sort()` only works when all items are of the same type.

Quick Revision — Lists

- Created with `[ ]`, items separated by commas.
- **Mutable**, index starts at **0**, last index = size – 1.
- Can hold **mixed data types**.
- `+` concatenates, `*` repeats.
- Methods: `append`, `extend`, `insert`, `remove`, `pop`, `clear`, `index`, `count`, `sort`, `reverse`, `copy`.

CHAPTER 21: Python Tuples

Definition: A **Tuple** is a compound data type similar to a List, but it is a **read-only list** — its values **cannot be changed or removed at all**. A tuple is created using **parentheses ()**.

Simple Meaning: A tuple is a list that has been locked. You can look at the values, but you cannot edit them.

21.1 Key points from the PDF

- A tuple is similar to a list.
- Tuples are **read-only lists** → values cannot be changed or removed.
- A **list** is created using **brackets []** and its elements and size **can** be changed.
- A **tuple** is created using **parentheses ()**.
- (PDF) A tuple cannot be printed with any other type of variable in a single print statement.

⚠ **PDF Check — Last point is misleading** The claim "Tuple can not be printed with any other type variable in single print statement" is **not correct as written**. This works perfectly:

```
t = (1, 2)
l = [3, 4]
print(t, l)      # Output: (1, 2) [3, 4]
```

What the slide really means is that you cannot **join (+)** a tuple with a different type:

```
print(t + l)      # x TypeError: can only concatenate tuple (not "list") to tuple
```

So the correct statement is: **a tuple can only be concatenated with another tuple**, not with a list or a string.

21.2 The PDF's Tuple Program

```
tuple = ('Ankit', 456, 1.14, 'Patel', 70.2, True, False)
tinytuple = (99, 'The Easylearn Academy')

print(tuple)          # Prints complete tuple
print(tuple[0])        # Prints first element
print(tuple[1:3])      # Prints elements at index 1 and 2
print(tuple[2:])       # Prints elements from index 2 to the end
print(tinytuple * 2)   # Prints tuple two times
print(tuple + tinytuple) # Prints concatenated tuple
```

Expected Output:

```
('Ankit', 456, 1.14, 'Patel', 70.2, True, False)
Ankit
(456, 1.14)
(1.14, 'Patel', 70.2, True, False)
(99, 'The Easylearn Academy', 99, 'The Easylearn Academy')
('Ankit', 456, 1.14, 'Patel', 70.2, True, False, 99, 'The Easylearn Academy')
```

What will NOT work:

```
tuple[0] = 'Patel'      # x TypeError – tuples are immutable
del tuple[0]            # x TypeError – cannot delete an item from a tuple
```

21.3 Tuple Methods

Since a tuple is read-only, you **cannot add or remove items**. Only **two** methods are available:

Method	What it does
<code>count(item)</code>	Counts how many times the specified item appears in the tuple
<code>index(item)</code>	Returns the index of the specified item

The PDF adds: if the item is **not found**, `index()` generates an error and **the program will stop**.

⚠ **PDF Check — Wrong error name** The PDF says `index()` raises a **KeyError**. In reality, `tuple.index()` (and `list.index()`) raise a **ValueError** when the item is not found. **KeyError** belongs to **dictionaries**. Write **ValueError** in your exam.

Example:

```
t = (10, 20, 30, 20)
print(t.count(20))    # 2
print(t.index(30))    # 2
print(t.index(99))    # x ValueError: tuple.index(x): x not in tuple
```

21.4 ★ List vs Tuple — Comparison Table (very common exam question)

Point	List	Tuple
Brackets used	Square brackets <code>[]</code>	Parentheses <code>()</code>
Mutability	Mutable (changeable)	Immutable (read-only)
Can add/remove items?	Yes	No
Size	Can change	Fixed
Number of methods	Many (append, insert, pop, sort...)	Only 2 (<code>count</code> , <code>index</code>)
Speed	Slower	Faster
Use when	Data will change	Data must stay constant
Example	<code>l = [1, 2, 3]</code>	<code>t = (1, 2, 3)</code>

Remember: List → `[ ]` → changeable. Tuple → `( )` → read-only, only `count()` and `index()`.

CHAPTER 22: Python Dictionary

Definition: A **Dictionary** is a collection created with **curly brackets** `{ }` that stores data as **key-value pairs**. Values are assigned and accessed using **square brackets** `[ ]` with the key.

Simple Meaning: A dictionary is like a real dictionary book: you look up a **word (key)** and get its **meaning (value)**. Instead of numbered positions, you use names to reach the data.

22.1 Key points from the PDF

- In Python, dictionaries are created with **curly brackets** `{ }` and have **keys and values**.
- One can think of the **key as the name of a variable** and the **value as the value of that variable**.
- One can **add a new key-value pair at any time**.
- Dictionaries are enclosed by **curly braces** `{ }`; values are assigned and accessed using **square braces** `[ ]`.
- Dictionaries can **retrieve values very fast when the key is known**.

Syntax:

```
dict_name = {"key1": value1, "key2": value2}
dict_name["key1"]          # access a value
dict_name["key1"] = new_value # change a value
dict_name["newkey"] = value  # add a new pair
del dict_name["key1"]        # delete a pair
```

⚠ **PDF Check — Two corrections on the definition slide 1.** The slide says "Python dictionary is one type of list which each value key which it is always string." Two problems: - A dictionary is **not** a list. It is a separate data type (a **mapping**) — a list uses **number positions**, a dictionary uses **keys**. - Keys are **not always strings**. A key can be any **immutable** value — string, number, or tuple. `{1: "one", 2: "two"}` is a perfectly valid dictionary. What keys **cannot** be is mutable (a list cannot be a key). 2. Write it this way instead: "A dictionary is an unordered collection of key-value pairs, where each key is unique and immutable."

22.2 Program 1 — The `teacher` dictionary (from the PDF)

```
teacher = {"name": "Ankit", "age": 38, "weight": 81.25, "gender": True,
           "country": "india", "secret": 123123}

print(teacher)          # print the whole dictionary

teacher['name'] = "ANKIT PATEL" # change a particular key's value
print(teacher["name"])    # print a specific key's value

del teacher["secret"]    # delete a specific key-value pair
print(teacher)
```

Expected Output:

```
{'name': 'Ankit', 'age': 38, 'weight': 81.25, 'gender': True, 'country': 'india', 'secret': 123123}
ANKIT PATEL
{'name': 'ANKIT PATEL', 'age': 38, 'weight': 81.25, 'gender': True, 'country': 'india'}
```

What this program demonstrates: 1. Creating a dictionary with mixed value types (string, int, float, bool). 2. **Changing** a value using its key. 3. **Reading** one value using its key. 4. **Deleting** a key-value pair using the `del` keyword.

Note about the PDF image: This slide is a screenshot whose right edge is **cut off in the PDF itself**, so the end of a few comment lines is not visible. The code above is the complete, working version of what the slide shows.

22.3 Program 2 — Building a dictionary step by step (from the PDF)

```

book = {}                                # Creating an empty Dictionary
print("Empty Dictionary: ")
print(book)

# Adding elements one at a time
book['name'] = 'secret'
book['price'] = 1000
book['weight'] = 1.5
print("Dictionary after adding 3 elements: ")
print(book)

# Adding set of values to a single Key
book['chapters'] = (1, 2, 3, 4)
book['topics'] = ['energy', 'focus', 'logical thinking']
print("Dictionary after adding 3 elements: ")
print(book)

# Updating existing Key's Value
book['price'] = 1200
print(book)

```

Expected Output:

```

Empty Dictionary:
{}
Dictionary after adding 3 elements:
{'name': 'secret', 'price': 1000, 'weight': 1.5}
Dictionary after adding 3 elements:
{'name': 'secret', 'price': 1000, 'weight': 1.5, 'chapters': (1, 2, 3, 4), 'topics': ['energy', 'focus', 'logical thinking']}
{'name': 'secret', 'price': 1200, 'weight': 1.5, 'chapters': (1, 2, 3, 4), 'topics': ['energy', 'focus', 'logical thinking']}

```

What this program demonstrates: 1. Creating an **empty dictionary** with `{}`. 2. Adding pairs **one at a time**. 3. A **value can itself be a tuple or a list** (a set of values under a single key). 4. Assigning to an **existing key updates** it instead of adding a new one.

⚠ **PDF Check — Two issues on this slide** 1. The slide writes `Book['topics'] = [...]` with a **capital B**, but the dictionary is named `book` (small b). Python is case-sensitive, so `Book` would give a `NameError`. It must be `book`. 2. The last part of the slide (`# Updating existing Key's Value`) is **cut off in the PDF image**. Based on the visible comment, the intended line is an update such as `book['price'] = 1200`, which is what is shown above.

22.4 Important Dictionary Methods

Method	What it does
<code>clear()</code>	Removes all items from the dictionary
<code>copy()</code>	Returns a shallow copy of the dictionary
<code>fromkeys(seq[, v])</code>	Returns a new dictionary with keys taken from <code>seq</code> and every value equal to <code>v</code> (default <code>None</code>)
<code>get(key[, d])</code>	Returns the value of the key. If the key does not exist, returns <code>d</code> (default <code>None</code>) — no error
<code>items()</code>	Returns a new object of the dictionary's items in (key, value) format
<code>keys()</code>	Returns a new object of the dictionary's keys
<code>pop(key[, d])</code>	Removes the item with that key and returns its value , or <code>d</code> if the key is not found. If <code>d</code> is not given and the key is missing → <code>KeyError</code>
<code>popitem()</code>	Removes and returns the last item (key, value). Raises <code>KeyError</code> if the dictionary is empty
<code>update([other])</code>	Adds elements from another dictionary. If a key is not present → it is added . If the key is present → its value is updated
<code>values()</code>	Returns a view object showing a list of all the values in the dictionary

`get()` vs `[ ]` — a very useful comparison

Point	<code>dict["key"]</code>	<code>dict.get("key")</code>
Key exists	Returns the value	Returns the value
Key missing	<code>KeyError</code> → program stops	Returns <code>None</code> (or your default) — no crash

Example:

```
student = {"name": "Pj", "marks": 88}

print(student.get("name"))      # Pj
print(student.get("city"))     # None
print(student.get("city", "N/A")) # N/A
print(student["city"])         # x KeyError
```

Example of the other methods:

```
student = {"name": "Pj", "marks": 88}

print(student.keys())          # dict_keys(['name', 'marks'])
print(student.values())        # dict_values(['Pj', 88])
print(student.items())         # dict_items([('name', 'Pj'), ('marks', 88)])

student.update({"marks": 95, "city": "Surat"})
print(student)                 # {'name': 'Pj', 'marks': 95, 'city': 'Surat'}

print(student.pop("city"))     # Surat
student.clear()
print(student)                 # {}
```

Remember: - Dictionary = `{ }`, key-value pairs, accessed with `[ ]`. - **Keys must be unique and immutable.** - Missing key with `[ ]` → **KeyError**; with `get()` → **None**. - Retrieval is **very fast when the key is known**.

CHAPTER 23: Sets

Definition: In Python, a **Set** is an **unordered collection of unique elements**. It is similar to a mathematical set, meaning it **does not allow duplicate values**.

Simple Meaning: A set is a bag of items with **no repeats** and **no fixed order**. If you put the same item in twice, it is kept only once.

23.1 Key Properties of Sets (4 points from the PDF)

#	Property	Meaning
1	Unordered	The order of elements is not guaranteed — items may print in a different order than you typed them
2	Unique	Duplicate elements are automatically removed
3	Mutable	You can add or remove elements
4	Immutable Elements	The elements inside a set must be of immutable types (numbers, strings, tuples)

In Simple Words (point 3 vs point 4 — this confuses many students): The **set itself can change** (you can add/remove items), but **each item inside must be an unchangeable type**. So a number, string or tuple can go inside a set — a **list cannot**.

```
s = {1, "hello", (2, 3)}    # ✓ all items are immutable
s = {1, [2, 3]}            # ✗ TypeError – a list is mutable, cannot go in a set
```

23.2 The PDF's Set Program

```
# Creating a set
fruits = {'apple', 'banana', 'orange'}
print(fruits)    # Output: {'banana', 'orange', 'apple'}

# Adding elements to a set
fruits.add('mango')
print(fruits)    # Output: {'banana', 'orange', 'apple', 'mango'}

# Removing an element from a set
fruits.remove('banana')
print(fruits)    # Output: {'orange', 'apple', 'mango'}

# Set operations
set1 = {1, 2, 3}
set2 = {2, 3, 4}

union = set1.union(set2)
print(union)     # Output: {1, 2, 3, 4}

intersection = set1.intersection(set2)
print(intersection) # Output: {2, 3}
```

What this program demonstrates:

Operation	Method	Meaning
Create	<code>{ }</code> with values	Makes a set
Add	<code>add(item)</code>	Adds one element
Remove	<code>remove(item)</code>	Removes one element
Union	<code>set1.union(set2)</code>	All elements from both sets (no duplicates)
Intersection	<code>set1.intersection(set2)</code>	Only the elements common to both sets

Remember (unordered!): The output order of a set may differ every time you run it. In the PDF's own output, `{'apple', 'banana', 'orange'}` printed as `{'banana', 'orange', 'apple'}`. **Never assume set order.**

Bonus point worth knowing: An empty set must be created with `set()`, **not** `{ }` — because `{ }` creates an empty **dictionary**.

```
a = {}          # this is an empty DICTIONARY
b = set()       # this is an empty SET
```

23.3 ★ Comparison Table — List vs Tuple vs Dictionary vs Set

Point	List	Tuple	Dictionary	Set
Symbol	[]	()	{ key: value }	{ }
Mutable?	Yes	No	Yes	Yes
Ordered / indexed?	Yes (0, 1, 2...)	Yes (0, 1, 2...)	Accessed by key	No order, no index
Duplicates allowed?	Yes	Yes	Keys: No / Values: Yes	No
Access an item	<code>l[0]</code>	<code>t[0]</code>	<code>d["key"]</code>	Cannot access by index
Example	[1, 2, 2]	(1, 2, 2)	{"a": 1}	{1, 2}

Quick Revision — Data Types (Chapters 17-23)

- **Immutable:** Number, String, Tuple → **Mutable:** List, Dictionary, Set.
- String: `str`, index from 0, `+` joins, `*` repeats, cannot change one character.
- List: `[ ]`, mixed types, mutable, many methods.
- Tuple: `( )`, read-only, only `count()` and `index()`.
- Dictionary: `{key: value}`, fast lookup by key, keys unique.
- Set: `{ }`, unordered + unique, `add()`, `remove()`, `union()`, `intersection()`.

CHAPTER 24: Multi-Line Statements

- Statements in Python **typically end with a new line**.
- Python allows the use of the **line continuation character** `\` to show that the **line should continue** on the next line.

Example from the PDF:

```
# Single line version
total = item_one + item_two + item_three

# OR - split over multiple lines
total = item_one + \
    item_two + \
    item_three
```

Both forms mean exactly the same thing.

In Simple Words: When one statement is too long to read comfortably, put a `\` at the end of the line and continue on the next line. Python then joins them into one statement.

Important rules: - Nothing (not even a space) may come **after the** `\` — it must be the very last character on that line. - Inside brackets `()`, `[]` or `{}`, the `\` is **not needed** — Python already knows the statement continues:

```
numbers = [1, 2, 3,
           4, 5, 6]    # works without a backslash
```

Remember: Line continuation character = **backslash** `\`.

CHAPTER 25: Operators in Python

Definition: Operators are **symbols** that perform operations on values or variables.

Simple Meaning: Operators are the signs (like `+`, `-`, `>`, and `and`) that tell Python to *do something* with your data.

Terminology: In `a + b`, the values `a` and `b` are called **operands**, and `+` is the **operator**.

25.0 Types of Operators (7 types listed in the PDF)

1. Arithmetic Operators
2. Comparison (Relational) Operators
3. Assignment Operators
4. Logical Operators
5. Bitwise Operators
6. Membership Operators
7. Identity Operators

Note: The PDF lists **Bitwise Operators** in this list but does **not** give a detailed slide for them. They appear only in the operator-precedence table at the end (`&`, `|`, `^`, `>>`, `<<`). So for exams based on this PDF, mention bitwise operators **by name** in the list of 7 types, and give the details for the other six.

25.1 Arithmetic Operators

(Examples in the PDF assume `a = 10`, `b = 20`.)

Operator	Name	Description	Example
<code>+</code>	Addition	Adds values on either side of the operator	<code>a + b = 30</code>
<code>-</code>	Subtraction	Subtracts the right operand from the left operand	<code>a - b = -10</code>
<code>*</code>	Multiplication	Multiplies values on either side	<code>a * b = 200</code>
<code>/</code>	Division	Divides the left operand by the right operand	<code>b / a = 2</code>
<code>%</code>	Modulus	Divides and returns the remainder	<code>b % a = 0</code>
<code>**</code>	Exponent	Performs a power calculation	<code>a ** b = 10 to the power 20</code>
<code>//</code>	Floor Division	Division where the digits after the decimal point are removed . If one operand is negative, the result is floored (rounded towards negative infinity)	<code>9 // 2 = 4</code> and <code>9.0 // 2.0 = 4.0</code>

`/` vs `//` vs `%` — the three that get confused

```
print(9 / 2)      # 4.5  → normal division, always gives a float
print(9 // 2)     # 4    → floor division, decimal part dropped
print(9 % 2)      # 1    → remainder
print(9.0 // 2.0) # 4.0  → floor division of floats gives a float
```

Negative floor division:

```
print(-9 // 2)   # -5 (NOT -4)
```

Because floor division always rounds **down** (towards negative infinity), `-4.5` becomes `-5`.

⚠ **PDF Check — Unclear example** The floor-division row's example column reads `9//2 = 4` and `9.0//2.0 = 4.0`, `-1`. The trailing `-1` is not explained anywhere and appears to be a leftover from a deleted negative-number example. Ignore it; use `-9 // 2 = -5` as your negative example instead. Also note the description says "*rounded away from zero (towards negative infinity)*" — the phrase "**towards negative infinity**" is the correct part; "*away from zero*" is only true for negative results.

Remember: `/` → decimal answer. `//` → whole-number answer (rounded down). `%` → remainder. `**` → power.

25.2 Comparison / Relational Operators

These compare two values and always give a result of **True** or **False**.

(PDF examples assume `a = 10`, `b = 20`.)

Operator	Description	Example	Result
<code>==</code>	True if the values of two operands are equal	<code>(a == b)</code>	not true (False)
<code>!=</code>	True if the values are not equal	<code>(a != b)</code>	True
<code>></code>	True if the left value is greater than the right	<code>(a > b)</code>	not true (False)
<code><</code>	True if the left value is less than the right	<code>(a < b)</code>	True
<code>>=</code>	True if the left value is greater than or equal to the right	<code>(a >= b)</code>	not true (False)
<code><=</code>	True if the left value is less than or equal to the right	<code>(a <= b)</code>	True

Example program:

```
a = 10
b = 20
print(a == b)  # False
print(a != b)  # True
print(a < b)   # True
print(a >= b)  # False
```

Remember (biggest beginner mistake): `=` is **assignment** (store a value). `==` is **comparison** (check if equal). Writing `if a = 5:` is a syntax error.

Point	<code>=</code>	<code>==</code>
Name	Assignment operator	Equality/comparison operator
Job	Stores a value into a variable	Checks whether two values are equal
Result	No True/False	Gives True or False
Example	<code>a = 5</code>	<code>a == 5</code>

25.3 Logical Operators

Used to **combine conditions**.

Operator	Name	Description (from PDF)	Example
<code>and</code>	Logical AND	If both operands are true, the condition becomes true	<code>(a == c and b == d)</code> is true
<code>or</code>	Logical OR	If any of the two operands is non-zero, the condition becomes true	<code>(a == c or b == d)</code> is true
<code>not</code>	Logical NOT	Used to reverse the logical state of its operand	<code>not(a == x and b == y)</code> is false

Truth tables (learn these — they answer most logical-operator questions)

AND — true only when **both** are true:

A	B	A and B
True	True	True
True	False	False
False	True	False
False	False	False

OR — true when **at least one** is true:

A	B	A or B
True	True	True
True	False	True
False	True	True
False	False	False

NOT — flips the value:

A	not A
True	False
False	True

Example program:

```
age = 20
marks = 75

print(age > 18 and marks > 60)  # True (both conditions true)
print(age > 25 or marks > 60)   # True (second condition true)
print(not(age > 18))            # False (reverses True)
```

Small clarification on `or`: The PDF describes `or` as "if any of the two operands are non-zero". In Python, values like `0`, `""`, `[]`, `{}` and `None` count as **False**; everything else counts as **True**. So "non-zero" really means "truthy".

Remember: `and` → all conditions must be true. `or` → any one is enough. `not` → reverses it.

25.4 Assignment Operators

Operator	Name	Description	Equivalent to
<code>=</code>	Assign	Assigns the value on the right to the variable on the left	<code>c = a + b</code> stores <code>a + b</code> into <code>c</code>
<code>+=</code>	Add AND	Adds the right operand to the left and assigns the result to the left	<code>c += a</code> → <code>c = c + a</code>
<code>-=</code>	Subtract AND	Subtracts the right operand from the left and assigns the result	<code>c -= a</code> → <code>c = c - a</code>
<code>*=</code>	Multiply AND	Multiplies and assigns the result	<code>c *= a</code> → <code>c = c * a</code>
<code>/=</code>	Divide AND	Divides and assigns the result	<code>c /= a</code> → <code>c = c / a</code>
<code>%=</code>	Modulus AND	Takes the modulus and assigns the result	<code>c %= a</code> → <code>c = c % a</code>
<code>**=</code>	Exponent AND	Performs a power calculation and assigns the result	<code>c **= a</code> → <code>c = c ** a</code>
<code>//=</code>	Floor Division AND	Performs floor division and assigns the result	<code>c //= a</code> → <code>c = c // a</code>

In Simple Words: These are **shortcuts**. `c += a` is just a shorter way of writing `c = c + a`.

Example program:

```
c = 10
a = 3

c += a    # c = 13
print(c)
c -= a    # c = 10
print(c)
c *= a    # c = 30
print(c)
c /= a    # c = 10
print(c)
c **= 2   # c = 100
print(c)
c %= 7    # c = 2
print(c)
```

Expected Output:

```
13
10
30
10
100
2
```

⚠ **PDF Check — Typo** The `/=` row in the PDF's table repeats itself: "`c /= a` is equivalent to `c = c / ac` `c /= a` is equivalent to `c = c / a`". It is just a copy-paste duplication. The correct statement is simply: **`c /= a` is equivalent to `c = c / a`**.

Remember: All compound assignment operators follow the same pattern: `c op= a` means `c = c op a`.

25.5 Membership Operators

Definition: Python's membership operators **test for membership in a sequence**, such as strings, lists or tuples.

Simple Meaning: They check: "Is this value present inside that collection?" The answer is **True** or **False**.

Operator	Description	Example
<code>in</code>	Evaluates to true if it finds the variable in the specified sequence, false otherwise	<code>x in y</code> → true if <code>x</code> is a member of sequence <code>y</code>
<code>not in</code>	Evaluates to true if it does not find the variable in the specified sequence, false otherwise	<code>x not in y</code> → true if <code>x</code> is not a member of <code>y</code>

The PDF's Membership Program (corrected)

```
a = 3
b = 20
list = [1, 2, 3, 4, 5]

isFound = a in list
print(isFound)

isFound = b not in list
print(isFound)

word = 'apple'
fruits = "banana mango pineapple orange apple"
isFound = word in fruits
print(isFound)
```

Expected Output:

```
True
True
True
```

Explanation: 1. `3 in [1,2,3,4,5]` → 3 is present → **True** 2. `20 not in [1,2,3,4,5]` → 20 is absent, so "not in" is → **True** 3. `'apple' in "banana mango pineapple orange apple"` → the text `apple` appears in the sentence → **True**

⚠ **PDF Check — Three errors in this slide's code** The slide as printed will **not run**. The mistakes: 1. `Word = 'apple'` but then `word in Fruits` — Python is case sensitive, so `word` and `Fruits` (defined as `Word` and `Fruits`) don't match up. `word` would raise a `NameError`. 2. `Print(isFound)` with a capital `P` → `NameError`. Python's function is lowercase `print()`. 3. **Curly quotes** (`'apple'`, `"banana..."`) instead of straight quotes → `SyntaxError`.

The version written above is the corrected, runnable one. Also note that `list = [...]` again shadows the built-in `list()`.

Remember: `in` works on **strings, lists, tuples, sets and dictionaries** (for dictionaries it checks the **keys**, not the values).

```
d = {"name": "Pj", "age": 21}
print("name" in d)    # True  (checks keys)
print("Pj" in d)      # False (values are not checked)
```

25.6 Identity Operators

Definition: Identity operators **compare the memory locations of two objects**.

Simple Meaning: They do not ask "*do these two have the same value?*" — they ask "*are these two the very same object in memory?*"

Operator	Description	Example
<code>is</code>	Evaluates to true if the variables on either side point to the same object , false otherwise	<code>x is y</code> → 1 (True) if <code>id(x)</code> equals <code>id(y)</code>
<code>is not</code>	Evaluates to false if the variables point to the same object , and true otherwise	<code>x is not y</code> → 1 (True) if <code>id(x)</code> is not equal to <code>id(y)</code>

The PDF's Identity Program

```
x = 10
y = 10
result = x is y
print("result:", result)
```

Output:

```
result: True
```

The PDF explains: "*We will get True because both x and y are identical.*"

25.7 The `id()` Function

Points from the PDF:

- We can check the id of variables using the `id()` function.
- The `id()` function **returns a unique id for a given object**.
- **Every object in Python gets a unique id when it is created.**
- The id of an object is an **integer value that represents the address of the object in memory**.

The PDF's `id()` example:

```
# variables
x = 10
y = 10
result = x is y
print("result:", result, id(x), id(y))
```

Sample Output (from the PDF):

```
result: True 4488129824 4488129824
```

Both ids are the **same number**, which proves `x` and `y` point to the **same object** in memory.

⚠ **PDF Check — An important caution about this example** The example works *only because Python pre-creates and reuses small integers* (roughly -5 to 256) to save memory. That is why `x = 10` and `y = 10` share one object. **This is not a general rule.** With larger numbers or with lists, the result is different:

```
a = 1000
b = 1000
print(a == b)    # True - same VALUE
print(a is b)    # may be False - different OBJECTS

l1 = [1, 2]
l2 = [1, 2]
print(l1 == l2)  # True
print(l1 is l2)  # False - two separate list objects
```

Also, the exact id numbers change every time you run the program, so your output will not match the PDF's numbers.

★ `is` vs `==` — Comparison Table (very commonly confused)

Point	<code>==</code>	<code>is</code>
Compares	Values	Memory addresses (identity)
Question it answers	"Do they hold the same data?"	"Are they literally the same object?"
Related function	—	<code>id()</code>
<code>l1 = [1,2]; l2 = [1,2]</code>	<code>l1 == l2</code> → True	<code>l1 is l2</code> → False
Use it for	Comparing data	Checking <code>None</code> , e.g. <code>if x is None:</code>

Remember: `==` compares values. `is` compares memory locations. This one sentence answers most 2-mark questions on identity operators.

Quick Revision — Operators

Type	Operators
Arithmetic	<code>+</code> <code>-</code> <code>*</code> <code>/</code> <code>%</code> <code>**</code> <code>//</code>
Comparison	<code>==</code> <code>!=</code> <code>></code> <code><</code> <code>>=</code> <code><=</code>
Assignment	<code>=</code> <code>+=</code> <code>-=</code> <code>*=</code> <code>/=</code> <code>%=</code> <code>**=</code> <code>//=</code>
Logical	<code>and</code> <code>or</code> <code>not</code>
Bitwise	<code>&</code> <code> </code> <code>^</code> <code>>></code> <code><<</code> <code>~</code> (listed but not detailed in the PDF)
Membership	<code>in</code> <code>not in</code>
Identity	<code>is</code> <code>is not</code>

CHAPTER 26: Operator Precedence

Definition: Operator precedence decides **which operator is evaluated first** when an expression contains more than one operator.

Simple Meaning: It is Python's version of BODMAS. In `2 + 3 * 4`, multiplication runs first, so the answer is `14`, not `20`.

26.1 Precedence Table (from the PDF — highest at the top)

Sr. No.	Operator	Description
1	<code>**</code>	Exponentiation (raise to the power)
2	<code>~ + -</code>	Complement, unary plus and unary minus
3	<code>* / % //</code>	Multiply, divide, modulo and floor division
4	<code>+ -</code>	Addition and subtraction
5	<code>>> <<</code>	Right and left bitwise shift
6	<code>&</code>	Bitwise AND
7	<code>^ </code>	Bitwise exclusive OR and regular OR
8	<code><= < > >=</code>	Comparison operators
9	<code><> == !=</code>	Equality operators
10	<code>= %= /= //= -= += *= **=</code>	Assignment operators
11	<code>is, is not</code>	Identity operators
12	<code>in, not in</code>	Membership operators
13	<code>not, or, and</code>	Logical operators

How to read this table: row 1 is done **first**, row 13 is done **last**.

Example:

```
print(2 + 3 * 4)      # 14  (* before +)
print((2 + 3) * 4)    # 20  (brackets first)
print(2 ** 3 ** 2)    # 512 (** works right to left: 2 ** 9)
print(10 - 4 - 3)     # 3   (- works left to right)
```

⚠ **PDF Check — Three points on this table** 1. `<>` is Python 2 only. It was the old "not equal to" operator and was **removed in Python 3**. Using `<>` in Python 3 gives a `SyntaxError`. Use `!=` only. 2. In real Python 3, **comparison, equality, identity (`is`) and membership (`in`) operators all share the SAME precedence level** — they are not four separate levels as shown in rows 8, 9, 11 and 12. 3. Row 13 groups `not`, `or`, `and` together, but they actually have **different** precedence among themselves: **`not` is highest, then `and`, then `or`**.

```
print(True or False and False)  # True - 'and' runs first
```

Reproduce the PDF's table if your paper follows this material, but knowing these three corrections will protect you in output-based questions.

Remember: The safest habit in real code: **use brackets** `()` whenever an expression mixes several operators. Brackets always win, and they make your intention obvious.

★ FINAL EXAM REVISION

Important Definitions

Term	Exam-friendly definition
Python	A general-purpose, interpreted, interactive, object-oriented and high-level programming language, developed by Guido van Rossum (1980–1990) in the Netherlands.
Comment	Text in a program that does not affect the outcome of the code; it is written only to explain what the program does.
Variable	A named location in main memory where data is stored temporarily. Each variable has a name, a value and a location.
Keyword	A reserved word with a special meaning that cannot be used as the name of a variable, class or function. Python has 35 keywords.
Data type	A definition of the type of data, e.g. <code>123</code> is integer data and <code>"hello"</code> is String data.
Mutable data type	A type in which part of the value can be changed — List, Dictionary, Set.
Immutable data type	A type in which part of the value cannot be changed, but the whole value can be replaced — Number, String, Tuple.
String	A sequence of characters; its data type in Python is <code>str</code> .
List	A special variable that stores multiple values inside <code>[]</code> , separated by commas; it is mutable and indexed from 0.
Tuple	A compound data type similar to a list, but read-only, created with <code>()</code> .
Dictionary	A collection of key–value pairs created with <code>{ }</code> , whose values are accessed using <code>[]</code> with the key.
Set	An unordered collection of unique elements that does not allow duplicate values.
Operator	A symbol that performs an operation on operands.
Membership operators	Operators (<code>in</code> , <code>not in</code>) that test whether a value is present in a sequence such as a string, list or tuple.
Identity operators	Operators (<code>is</code> , <code>is not</code>) that compare the memory locations of two objects.
<code>id()</code> function	A function that returns a unique integer id representing the memory address of an object.
Operator precedence	The order in which operators are evaluated when an expression has more than one operator.
Line continuation character	The backslash <code>\</code> , used to continue one statement onto the next line.

Important Syntax

```

# Comments
# single line comment
'''
multi-line comment
'''

# Printing
print("message")
print(variable)
print("message", variable)
print("text", end='')          # stay on the same line

# Quotes
'single'  "double"  """triple for multi-line"""

# Multiple statements on one line
print("Hi"); print("Bye")

# Variable
variable_name = value

# Input
variable = input("input message")
number = int(input("Enter a number: "))  # convert to int

# Line continuation
total = a + \
        b

# String
word = "Hello"
word[0]      # first character
word[0:5]    # slice: index 0 to 4
word[2:]     # index 2 to end
word + " TEST" # concatenation
word * 2      # repetition

# List
my_list = ['Ankit', 123, 2.23, True]
my_list[0]      # access
my_list[1:3]    # slice
my_list.append(item)
my_list.insert(pos, item)
my_list.remove(item)
my_list.pop(pos)
my_list.sort()
my_list.reverse()

# Tuple
my_tuple = ('Ankit', 456, 1.14)
my_tuple.count(item)
my_tuple.index(item)

# Dictionary
teacher = {"name": "Ankit", "age": 38}
teacher["name"]      # access
teacher["name"] = "ANKIT" # change
teacher["city"] = "Surat" # add
del teacher["age"]    # delete
teacher.get("key", default)
teacher.keys(); teacher.values(); teacher.items()
teacher.update({"age": 40})

# Set
fruits = {'apple', 'banana'}
fruits.add('mango')
fruits.remove('banana')
set1.union(set2)
set1.intersection(set2)
empty = set()          # NOT {}

# Operators
+ - * / % ** //      # arithmetic
== != > < >= <=     # comparison
= += -= *= /= %= **= // = # assignment
and or not           # logical
in not in            # membership
is is not            # identity
id(x)                # memory id

```

Important Concepts You Must Understand

1. **Interpreted vs compiled** — Python runs line by line; no separate compile step needed.

2. **Dynamic typing** — no type declaration; Python decides the type from the value.
3. **Case sensitivity** — applies to variables, keywords and function names (`print` $\neq$ `Print`).
4. **Mutable vs immutable** — the single most important classification (NST fixed / LDS changeable).
5. **Indexing and slicing** — index starts at 0, last index = size – 1, and in `[start:end]` the **end is excluded**.
6. `input()` **always returns a string** — convert with `int()` / `float()`.
7. `=` **vs** `==` — assignment vs comparison.
8. `is` **vs** `==` — memory identity vs value equality.
9. **Sets are unordered and unique** — never depend on their printing order.
10. **Dictionary keys are unique**; lookup is very fast when the key is known.
11. **Operator precedence** — `**` first, logical operators last; brackets override everything.
12. `end=''` controls what `print()` puts at the end instead of a new line.

Important Programs from the PDF

#	Program	What it demonstrates
1	Hello World (<code>print("Hello Student")</code>)	First program, <code>print()</code> , single-line and multi-line comments
2	Quotation example	Single, double and triple quotes; multi-line strings
3	<code>print("Hi"); print("I am learning Python")</code>	Multiple statements on one line using <code>;</code>
4	Variable creation (<code>Name</code> , <code>Year</code> , <code>Weight</code> , <code>Gender</code>)	Storing string, int, float and boolean without declaring types
5	Printing with a variable	Printing a variable alone and together with a message
6	String program (<code>word = 'Hello World!'</code>)	Indexing, slicing, <code>*</code> repetition, <code>+</code> concatenation, immutability error
7	List program (<code>list = ['Ankit', 123, 2.23, 'Patel', True]</code>)	Mixed data types, indexing, slicing, <code>*</code> , <code>+</code>
8	Tuple program (<code>tuple = ('Ankit', 456, ...)</code>)	Same operations as a list, but read-only
9	Dictionary — teacher	Creating, changing a value, reading by key, deleting with <code>del</code>
10	Dictionary — book	Empty dictionary, adding items one by one, a tuple/list as a value, updating a key
11	Set program — fruits , <code>set1</code> , <code>set2</code>	Creating, <code>add()</code> , <code>remove()</code> , <code>union()</code> , <code>intersection()</code>
12	Membership example (<code>a in list</code> , <code>word in Fruits</code>)	<code>in</code> and <code>not in</code> on a list and on a string
13	Identity example (<code>x is y</code> with <code>id()</code>)	<code>is</code> operator and the <code>id()</code> function showing the memory address
14	Multi-line statement (<code>total = item_one + \</code>)	Line continuation using <code>\</code>

△ Commonly Confused Concepts

Confusion	The clear difference
<code>=</code> vs <code>==</code>	<code>=</code> stores a value; <code>==</code> checks equality
<code>is</code> vs <code>==</code>	<code>is</code> compares memory address; <code>==</code> compares values
<code>/</code> vs <code>//</code>	<code>/</code> gives a decimal result; <code>//</code> drops the decimal part (rounds down)
<code>%</code> vs <code>//</code>	<code>%</code> gives the remainder ; <code>//</code> gives the quotient
List vs Tuple	<code>[]</code> mutable, many methods vs <code>()</code> immutable, only 2 methods
Dictionary vs Set	Both use <code>{ }</code> ; a dictionary has key: value pairs, a set has only values
<code>{}</code> vs <code>set()</code>	<code>{}</code> creates an empty dictionary ; <code>set()</code> creates an empty set
Mutable vs Immutable	Part changeable (List, Dict, Set) vs part fixed (Number, String, Tuple)
<code>append()</code> vs <code>extend()</code>	<code>append()</code> adds one item; <code>extend()</code> adds all items of another list
<code>remove()</code> vs <code>pop()</code>	<code>remove()</code> takes the value ; <code>pop()</code> takes the position and returns the item
<code>dict["key"]</code> vs <code>dict.get("key")</code>	Missing key → <code>[]</code> gives KeyError ; <code>get()</code> gives None
Single vs multi-line comment	<code>#</code> for one line; <code>''' '''</code> for many lines
POP vs OOP	POP = function-based; OOP = class/object-based; Python supports both
Open source vs Free	Open source = code is public; Free = costs nothing
Slice end index	<code>[0:5]</code> gives index 0 to 4 — the end is always excluded

Last-Minute Revision (read this 15 minutes before the exam)

Basics - Python = general-purpose, interpreted, interactive, object-oriented, high-level. - Guido van Rossum, 1980–1990, Netherlands, named after **Monty Python's Flying Circus**. - 8 advantages: Readable, Easy, Cross-platform, Open source, Large library, Free, Exception handling, Automatic memory management. - Website: python.org • Docs in **HTML, PDF, PostScript**. - 7 applications: Web (Django/Flask), ML, Data Analysis, Scripting, Games, Embedded, Desktop (TKinter/QT).

Writing code - Comments: `#` single line, `''' '''` multi-line. - Quotes: `' '`, `" "`, `''' '''` (multi-line). Same quote must open and close. - `;` joins simple statements only — never `if`, loops, functions or classes. - Line continuation character = `\`.

Variables - Named memory location: **name + value + location**. - No type declaration; Python infers the type. - Case sensitive; cannot be a keyword; must start with a letter or `_`. - **35 keywords**, all case sensitive, `True/False/None` capitalised.

Input - `variable = input("message")` → **always returns a string** → wrap in `int()` / `float()` for numbers.

Data types - **Immutable: Number, String, Tuple. Mutable: List, Dictionary, Set.** - String → `str`, index from 0, `+` joins, `*` repeats, cannot edit one character. - List → `[ ]`, mixed types, mutable, methods: `append`, `extend`, `insert`, `remove`, `pop`, `clear`, `index`, `count`, `sort`, `reverse`, `copy`. - Tuple → `( )`, read-only, only `count()` and `index()` (missing item → **ValueError**). - Dictionary → `{key: value}`, `[ ]` to access, `del` to delete, `get()` avoids `KeyError`, fast lookup. - Set → `{ }`, **unordered + unique**, `add()`, `remove()`, `union()`, `intersection()`.

Operators — 7 types Arithmetic • Comparison • Assignment • Logical • Bitwise • Membership • Identity

- `+` `-` `*` `/` `%` `**` `//` — remember `9/2 = 4.5`, `9//2 = 4`, `9%2 = 1`.
- `==` `!=` `>` `<` `>=` `<=` — result is always `True/False`.
- `and` (all true) • `or` (any true) • `not` (reverse).
- `c += a` means `c = c + a` (same pattern for all compound operators).
- `in` / `not in` → membership in a sequence.
- `is` / `is not` → compares **memory address**; `id()` returns the unique memory id.
- Precedence: `**` first → arithmetic → comparison → logical last. **Brackets always win.**

Careful with (from PDF errors) - `<>` does not exist in Python 3 — use `!=`. - `print` is lowercase; `Print` gives an error. - Curly quotes “ ” from slides will not run — always use straight quotes. - Tuple `index()` raises **ValueError**, not `KeyError`. - Dictionary keys need not be strings — they must be **immutable**. - `x is y` being `True` for `10` works only because Python caches small integers.

Summary of All PDF Issues Flagged in These Notes

Slide topic	Issue	Correct version
Introduction	Says Python is under GNU GPL	Python uses the PSF License (still free/open source)
Multiple statements	" <i>but it is not possible</i> " reads as a contradiction	Means: not possible for if/loops/functions/classes
Quotation & several code slides	Curly quotes from PowerPoint	Must use straight <code>'</code> and <code>"</code>
<code>end=''</code> slide	Mentions <code>end=''</code> but the example never uses it; calls it "the second argument"	<code>end</code> is a keyword argument : <code>print("x", end='')</code>
Keywords	" <i>type keywords</i> " in the Python shell	Use <code>help("keywords")</code> or <code>keyword.kwlist</code>
<code>input()</code>	" <i>accepts any type of value</i> "	Always returns a string ; convert with <code>int()</code> / <code>float()</code>
String slide	<code>word[0:5]</code> described as "0th to 2nd position"; stray output row; <code>easylern</code> typo	<code>word[0:5]</code> = index 0 to 4 = <code>Hello</code>
List slide	<code>list[1:3]</code> described as "1st till 3rd"; variable named <code>list</code>	Gives index 1 and 2 only; avoid shadowing built-ins
Tuple slide	" <i>cannot be printed with any other type variable</i> "	You can print them together; you cannot concatenate different types
Tuple methods	<code>index()</code> raises KeyError	It raises ValueError
Dictionary slide	" <i>one type of list</i> ", " <i>key is always string</i> "	It's a mapping, not a list; keys are any immutable type
Dictionary <code>book</code> code	<code>Book['topics']</code> capital B; slide image cut off	Must be <code>book['topics']</code>
Dictionary/Set/ <code>teacher</code> code slides	Right edge of screenshots cropped inside the PDF itself	Complete versions reconstructed in Chapters 22 and 23
Assignment operators	<code>/=</code> row text duplicated	<code>c /= a</code> is equivalent to <code>c = c / a</code>
Membership example	<code>Word/word</code> case mismatch, <code>Print()</code> capital P, curly quotes	Corrected version given in Chapter 25.5
Floor division	Example ends with an unexplained <code>, -1</code>	Use <code>-9 // 2 = -5</code> as the negative example
Identity/ <code>id()</code>	Implies <code>is</code> is always True for equal numbers	Only true for cached small integers (about -5 to 256)
Precedence table	Lists <code><></code> ; splits comparison/identity/membership into separate levels; groups <code>not or and</code>	<code><></code> is Python 2 only; those three share one level; <code>not > and > or</code>
Types of Operator	Bitwise operators listed but never explained	Only appear in the precedence table (<code>& ^ >> <<</code>)

End of notes — all 50 slides of the PDF are covered. ✓

Tip: practise every program in these notes by typing it yourself. Reading Python and writing Python build two different skills, and exams test the second one.